

Knoweb

Sigla do Projeto - Nome do Projeto

Documento de Arquitetura de Software

Versão 1.0

Histórico de Revisões			
Versão	Data	Descrição	Autor
1.0	14/05	Feito até o 6.2	Kauan Henrique
2.0	21/05	Feito até o 14	Kauan Henrique
3.0	28/05	Atualizações	Kauan, Nicholas

Sumário

1. INTRODUÇÃO	4
2. IDENTIFICAÇÃO DO PROJETO	4
3. REPRESENTAÇÃO ARQUITETURAL	4
4. METAS E RESTRIÇÕES DA ARQUITETURA	4
5. VISÃO DE CASOS DE USO	5
5.1. Realizações de Casos de Uso	5
6. VISÃO LÓGICA	5
6.1. Visão Geral	5
6.2. Pacotes de Design Significativos do Ponto de Vista da Arquitetura	5
6.3. Camadas	5
7. VISÃO DE PROCESSOS	6
8. VISÃO DE IMPLANTAÇÃO	6
9. VISÃO DA IMPLEMENTAÇÃO	6
9.1. Visão Geral	6
10. VISÃO DE DADOS (OPCIONAL)	6
11. TAMANHO E DESEMPENHO	7
12. QUALIDADE	7
13. ANEXOS	7
14. REFERÊNCIAS	7
15. APROVAÇÕES	7

Documento de Arquitetura de Software

1. Introdução

Este documento descreve a arquitetura do sistema de e-commerce, abrangendo suas principais funcionalidades, componentes e decisões técnicas. O objetivo é fornecer uma visão clara da estrutura do sistema para desenvolvedores, stakeholders e equipe de projeto.

2. Identificação do Projeto

Projeto	Site de E-commerce
Requisitante	Renata
Gerente de Projetos	Kauan Henrique, Nicholas Milanin, Guilherme Barbosa

3. Representação Arquitetural

O sistema será representado através das seguintes visões:

- **Visão de Casos de Uso:** Funcionalidades do cliente e administrador.
- **Visão Lógica:** Componentes e pacotes do sistema.
- **Visão de Implementação:** Interfaces e camadas técnicas.
- **Visão do Processo:** Concorrência, sincronização e comunicação.
- **Visão Física:** Topologia de hardware e mapeamento do software para o hardware.

4. Metas e Restrições da Arquitetura

Metas:

- **Usabilidade:** Interface intuitiva (ex: auto-preenchimento de busca, menu de categorias) Normas com base na: **ISO 9241-11**
- **Segurança:** Alto nível para cadastros, pagamentos e dados de clientes. Normas com base na:

ISO 2700X
- **Desempenho:** Suporte a alta concorrência e sincronização em tempo real.

- **Resiliência e Tolerância a Falhas:** O sistema deve ser capaz de continuar funcionando mesmo que um ou mais componentes falhem. Isso inclui mecanismos como circuit breakers, retries, isolamento de falhas e comunicação assíncrona.
- **Manutenibilidade e Evolução Independente:** Cada microserviço deve ser pequeno, focado em uma única responsabilidade de negócio e capaz de ser desenvolvido, implantado e atualizado de forma independente, minimizando o impacto em outras partes do sistema.
- **Eficiência de Desenvolvimento:** Fornecer uma arquitetura que suporte um ciclo de desenvolvimento ágil, com ambientes de desenvolvimento consistentes, CI/CD automatizado e ferramentas que facilitem o trabalho dos desenvolvedores.
-

Restrições:

- **Tecnologia:**
 - o **Frontend:** Limitado a **React, Angular ou Vue.js**.
 - o **Backend (Microserviços):** Desenvolvido exclusivamente em **Java ou Python**.
 - o **Banco de Dados:** Principalmente **MySQL**, com exceções justificadas e aprovadas.
 - o **Comunicação Assíncrona (Barramento de Mensagens), Descoberta de Serviços, Configuração Centralizada e Gateway de API:** Soluções compatíveis com o ecossistema **Java/Python** deverão ser escolhidas.
 - o **Monitoramento e Log:** Ferramentas que se integrem bem com serviços Java/Python (e.g., Prometheus, Grafana, ELK Stack).
- **Prazos:**
 - o **Cronograma:** Rigorosa conformidade com o **cronograma do projeto**.
 - o **Reuniões:** Participação obrigatória em **reuniões semanais** de alinhamento e status.
 - o **Entregas:** Projeto dividido em **iterações curtas (sprints)** com entregas incrementais.
- **Qualidade e Processo:**
 - o **Testes:** Cobertura abrangente de testes (unitários, integração, ponta a ponta).
 - o **Documentação:** Manter documentação técnica **atualizada**.
 - o **Controle de Versão:** Uso de **Git** para todo o código.
 - o **Padrões de Código:** Definição e seguimento de **padrões de codificação**.
 - o **Ambientes:** Processo claro para gestão de ambientes (desenvolvimento, homologação, produção).
 - o **Segurança:** Prioridade para as **melhores práticas de segurança** em todas as camadas.

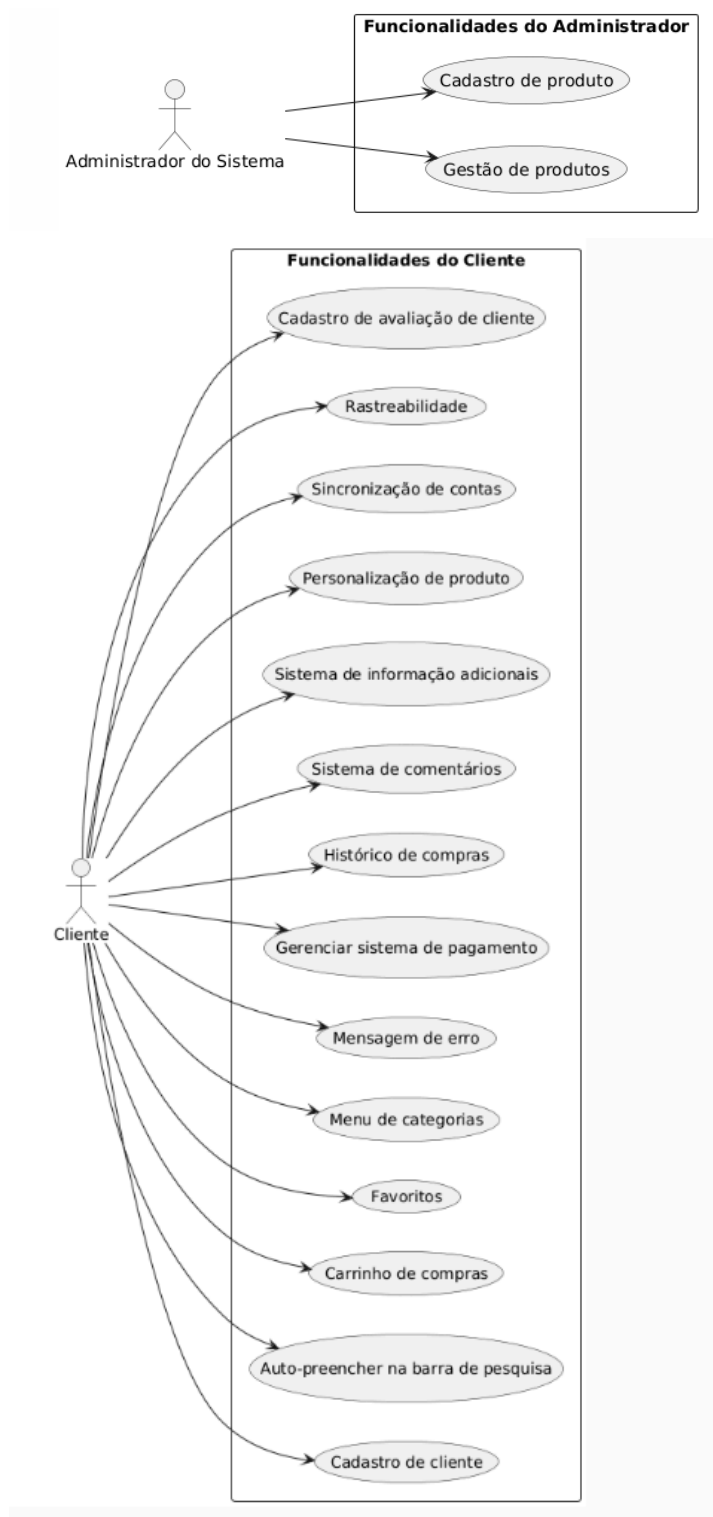
5. Visão de Casos de Uso

A **Visão de Casos de Uso** é uma parte fundamental do Documento de Arquitetura de Software que descreve **como os usuários interagem com o sistema** e quais funcionalidades são essenciais para atender às necessidades do negócio. Ela ajuda a entender:

- **Quem** usa o sistema (atores).
- **O que** o sistema deve fazer (funcionalidades).

- Como essas interações acontecem (fluxos).

1.1. Realizações de Casos de Uso

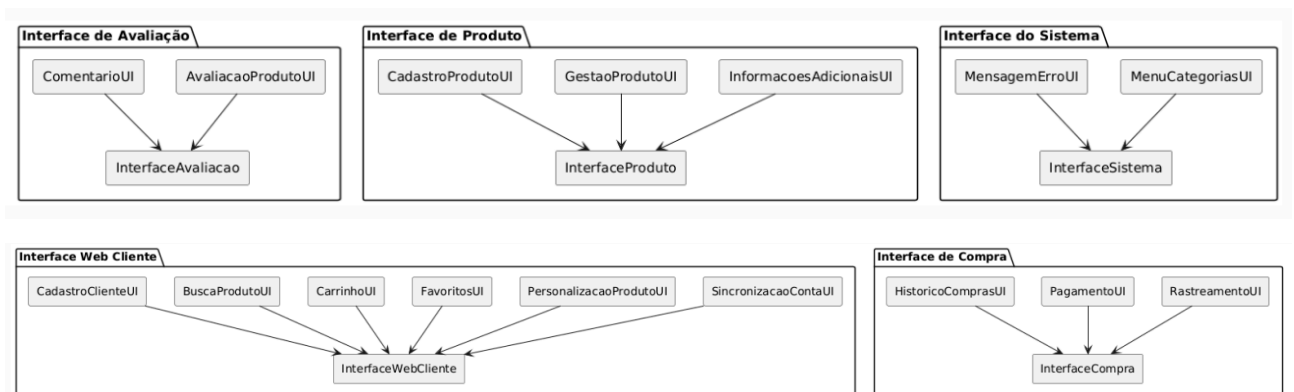


6. Visão Lógica

Definição

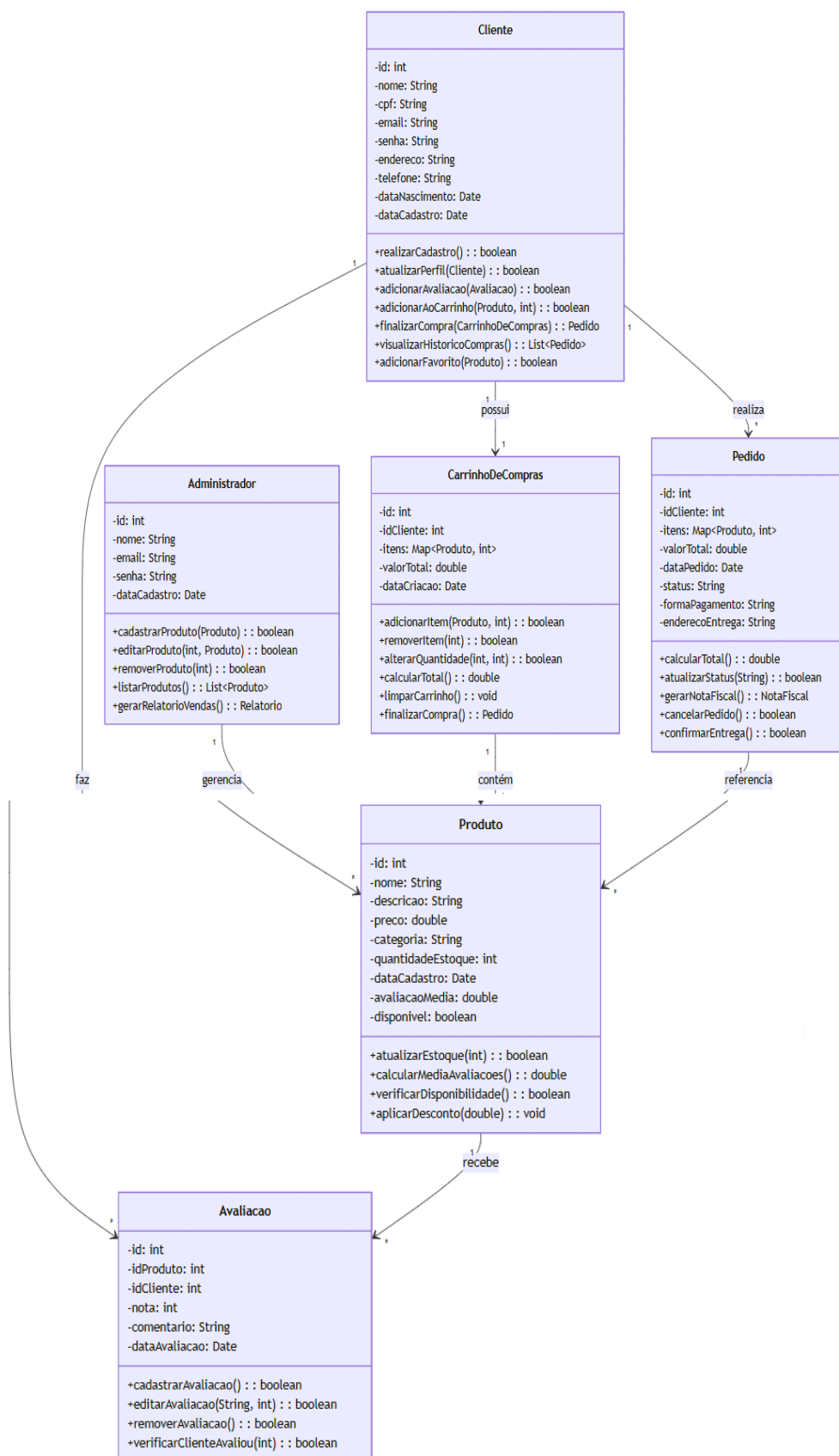
A Visão Lógica é uma representação arquitetural que descreve a estrutura interna do software do ponto de vista de sua organização em componentes lógicos. Ela responde à pergunta: *"Como o sistema está estruturado em termos de módulos, classes e suas interações?"*

1.2. Visão Geral



1.3. Pacotes de Design Significativos do Ponto de Vista da Arquitetura





1.4. Camadas

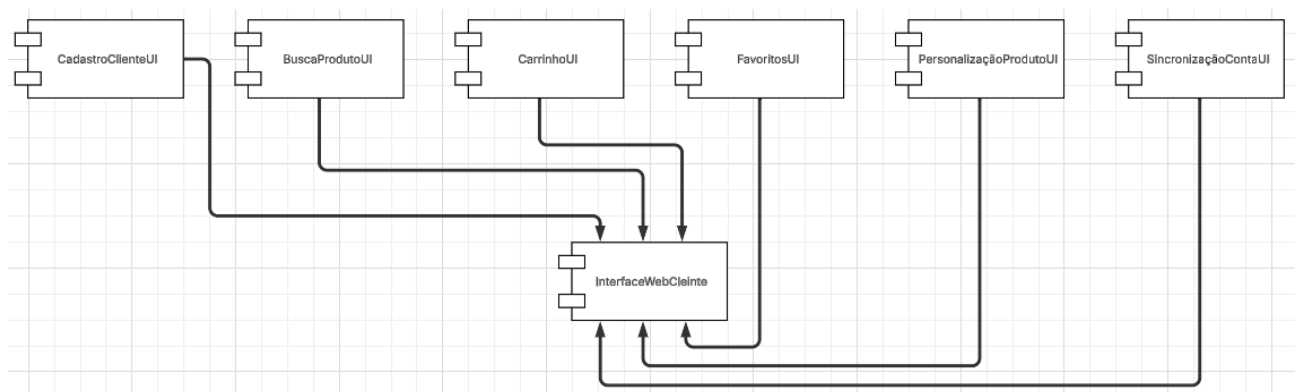
Arquitetura Model-View-Controller (MVC) O padrão arquitetural MVC é amplamente utilizado no desenvolvimento de software para separar a lógica de negócios, a apresentação dos dados e a interação do usuário em três componentes interconectados:

- **Model:** Representa os dados e a lógica de negócios da aplicação. É responsável por gerenciar o estado dos dados, as regras de negócios e a persistência. O Model é independente da interface do usuário.
- **View:** É a camada de apresentação, responsável por exibir os dados do Model para o usuário e por receber as interações do usuário. A View não contém lógica de negócios e deve ser o mais "burra" possível, apenas refletindo o estado do Model.
- **Controller:** Atua como um intermediário entre o Model e a View. Ele recebe as entradas do usuário da View, processa essas entradas (muitas vezes interagindo com o Model para atualizar dados ou executar lógica de negócios) e, em seguida, atualiza a View com os resultados.

Camada da View A camada da View é crucial para a interação do usuário com o sistema. Ela é composta por diversas interfaces de usuário (UI) que exibem informações e permitem a entrada de dados. As imagens fornecidas ilustram componentes específicos da camada da View, agrupados por funcionalidades ou módulos.

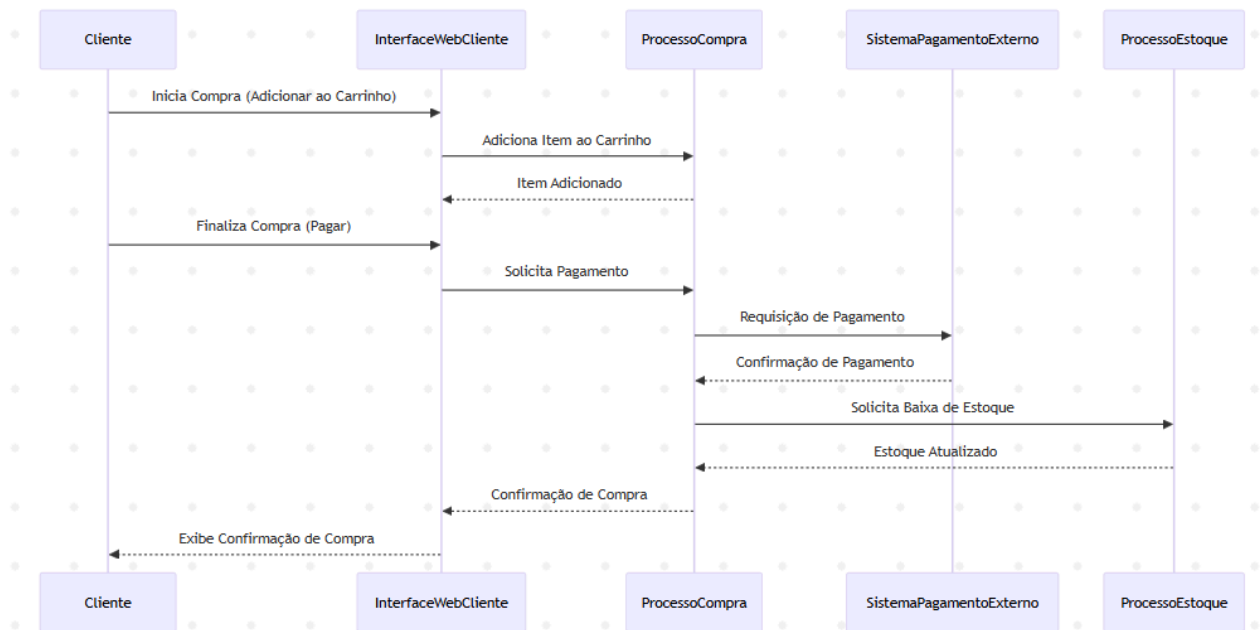
Diagrama de Componentes da Camada da View

Interface Web Cliente



7. Visão de Processos

Esta seção descreve a decomposição do sistema em processos leves (threads simples de controle) e processos pesados (agrupamentos de processos leves). A organização desta seção será feita em grupos de processos que se comunicam ou interagem. Serão descritos os modos principais de comunicação entre processos, como transmissão de mensagens e interrupções.



8. Visão de Implantação

Configuração de Rede Física e Implantação de Software

Para o sistema, a arquitetura de implantação proposta visa garantir alta disponibilidade, escalabilidade e desempenho. A configuração da rede física envolverá servidores de aplicação, servidores de banco de dados e um balanceador de carga para distribuir o tráfego de requisições.

1. Ambiente de Infraestrutura

- **Cloud ou On-Premise:** O sistema será preferencialmente implantado em um ambiente de **nuvem** (e.g., AWS, Google Cloud, Azure) para aproveitar sua escalabilidade elástica, gerenciamento de serviços e ferramentas de CI/CD. Em alternativa, se houver restrições, pode ser considerado um ambiente on-premise com tecnologias de virtualização e orquestração de contêineres.
- **Orquestração de Contêineres:** Utilizaremos **Kubernetes** (K8s) como a plataforma de orquestração de contêineres. Isso permitirá a implantação, escalonamento automático, gerenciamento de ciclo de vida e autorrecuperação dos microsserviços. Cada microsserviço será empacotado como um contêiner Docker.
- **Rede:** Será configurada uma **rede privada virtual (VPC/VNet)** para isolar o ambiente do sistema, com sub-redes para diferentes tipos de recursos (e.g., microsserviços, bancos de dados, barramento de mensagens). Regras de firewall e grupos de segurança serão aplicados para controlar o tráfego entre os componentes.

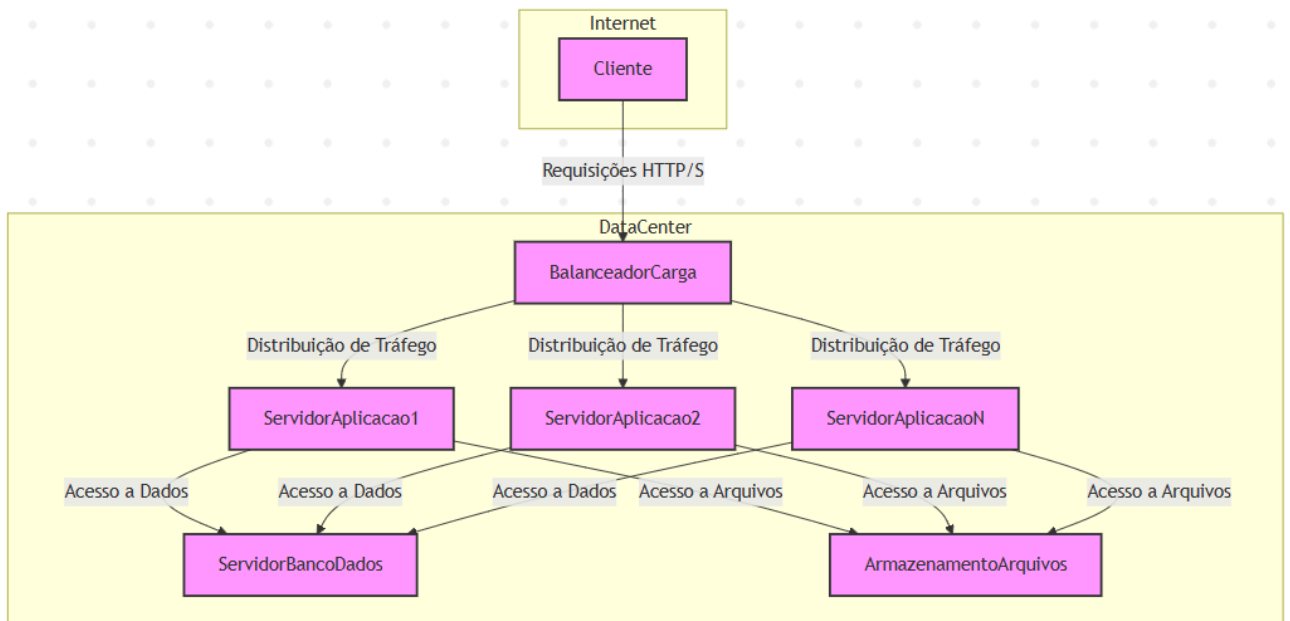
2. Componentes do Sistema e Implantação

- **Frontend (SPA - React/Angular/Vue.js):**
 - o A aplicação SPA será construída e seus arquivos estáticos (HTML, CSS, JavaScript) serão servidos através de um **serviço de armazenamento de objetos** (e.g., Amazon S3, Google Cloud Storage) ou um servidor web leve (e.g., Nginx) por trás de um CDN (Content Delivery Network) para alta disponibilidade e baixa latência global.
 - o As requisições da SPA para o backend passarão pelo **Gateway de API**.
- **Gateway de API (Java/Python):**
 - o Implantado como um **microsserviço dedicado no Kubernetes**, com alta disponibilidade (múltiplas instâncias).

- o Configurado para rotear requisições para os microsserviços apropriados, além de lidar com autenticação, autorização (possivelmente integrado a um Identity Provider externo), limitação de taxa e circuit breaking.
- **Microsserviços (Java/Python):**
 - o Cada microsserviço será implantado como um **deployment independente no Kubernetes**. Isso permite que cada um seja escalado independentemente, atualizado sem afetar os outros e gerencie seus próprios recursos.
 - o As imagens Docker de cada microsserviço serão armazenadas em um **repositório de contêineres privado** (e.g., Docker Hub privado, Amazon ECR, Google Container Registry).
- **Serviço de Descoberta (e.g., Eureka/Consul):**
 - o Será implantado como um **cluster de alta disponibilidade no Kubernetes**.
 - o Todos os microsserviços e o Gateway de API se registrarão e consultarão este serviço para encontrar as instâncias ativas dos outros serviços.
- **Barramento de Mensagens (e.g., Apache Kafka/RabbitMQ):**
 - o Um **cluster altamente disponível** do barramento de mensagens será implantado e gerenciado.
 - o Microsserviços se comunicarão assincronamente através de tópicos/filas, desacoplando suas dependências e melhorando a resiliência.
- **Configuração Centralizada (e.g., Spring Cloud Config):**
 - o Implantado como um **microsserviço centralizado**, geralmente também em um cluster de alta disponibilidade.
 - o Ele servirá configurações para todos os outros microsserviços, permitindo atualizações dinâmicas de configuração sem a necessidade de reimplantar os serviços.
- **Bancos de Dados (MySQL):**
 - o Cada microsserviço terá sua **própria instância de banco de dados MySQL**, garantindo independência de dados e escalabilidade.
 - o Serão utilizadas **instâncias gerenciadas de banco de dados** na nuvem (e.g., Amazon RDS MySQL, Google Cloud SQL for MySQL) para aproveitar recursos como backups automáticos, replicação, alta disponibilidade e patches de segurança.
 - o Acesso aos bancos de dados será restrito apenas aos microsserviços correspondentes e componentes de gerenciamento.
- **Monitoramento e Log (e.g., Prometheus/Grafana, ELK Stack):**
 - o **Coleta de Métricas:** Agentes de monitoramento serão configurados em cada contêiner de microsserviço para coletar métricas de desempenho (CPU, memória, latência, erros) e enviá-las para um sistema de agregação (e.g., Prometheus). As métricas serão visualizadas em painéis (e.g., Grafana).
 - o **Coleta de Logs:** Os logs gerados por todos os microsserviços serão centralizados em uma plataforma de log (e.g., ELK Stack - Elasticsearch para armazenamento, Logstash para processamento, Kibana para visualização).
 - o **Tracing Distribuído:** Ferramentas de tracing (e.g., Jaeger, Zipkin) serão integradas para rastrear requisições através de múltiplos microsserviços, facilitando a depuração em um ambiente distribuído.

3. Processo de Implantação e Operações

- **CI/CD (Integração Contínua/Entrega Contínua):** Um pipeline de CI/CD será implementado para automatizar a construção, teste e implantação dos microsserviços.
 - o **Integração Contínua:** Toda mudança de código acionará automaticamente a compilação, execução de testes unitários e de integração, e a criação de imagens Docker.
 - o **Entrega Contínua:** Após a validação na CI, as imagens Docker serão enviadas para o repositório de contêineres. A implantação nos ambientes de homologação e produção será automatizada através de ferramentas de orquestração e gerenciamento de configuração.
- **Estratégias de Implantação:** Utilizaremos estratégias de implantação seguras, como **Rolling Updates** (atualização gradual das instâncias) ou **Blue/Green Deployments** (implantação de uma nova versão em paralelo antes de desviar o tráfego), para minimizar o tempo de inatividade e os riscos.
- **Alertas e Notificações:** Serão configurados alertas proativos com base nas métricas e logs para notificar a equipe de operações sobre anomalias ou problemas no sistema.



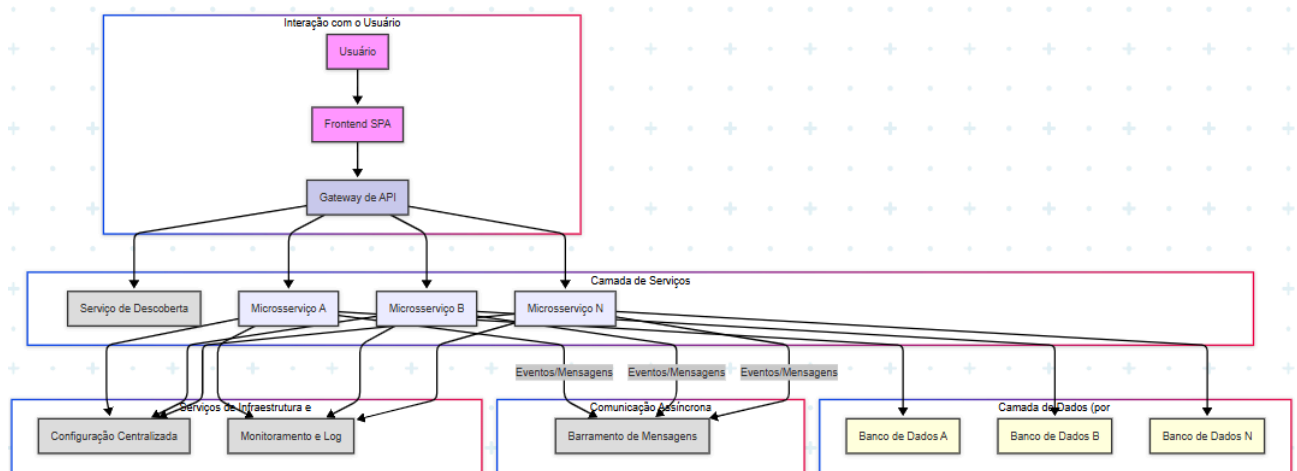
9. Visão da Implementação

a Visão da Implementação detalha a organização interna do código, mostrando como ele é estruturado em módulos e camadas para facilitar o desenvolvimento, a manutenção e a evolução do sistema.

1.5. Visão Geral

- **Gateway de API (API Gateway):** Ponto de entrada unificado para todas as requisições externas. Ele roteia as requisições para os microsserviços apropriados, lida com autenticação, autorização, limitação de taxa e outras preocupações transversais.
- **Microsserviços (Microservices):** O sistema será decomposto em um conjunto de serviços pequenos e independentes, cada um responsável por uma funcionalidade de negócio específica (e.g., Serviço de Usuários, Serviço de Produtos, Serviço de Pedidos, Serviço de Pagamento, Serviço de Avaliação). Cada microsserviço terá seu próprio banco de dados (se necessário) e será desenvolvido e implantado de forma independente.
- **Serviço de Descoberta (Discovery Service):** Permite que os microsserviços se registrem e se descubram mutuamente. Isso é essencial para que o Gateway de API e outros microsserviços possam encontrar e se comunicar com os serviços necessários de forma dinâmica.
- **Barramento de Mensagens (Message Broker/Queue):** Utilizado para comunicação assíncrona entre microsserviços. Isso desacopla os serviços, permitindo que eles se comuniquem por meio de eventos, melhorando a resiliência e a escalabilidade.
- **Configuração Centralizada (Centralized Configuration):** Um serviço para gerenciar as configurações de todos os microsserviços em um local centralizado, facilitando a atualização e o gerenciamento de configurações em diferentes ambientes.
- **Monitoramento e Log (Monitoring & Logging):** Ferramentas para coletar métricas, logs e traces de todos os microsserviços, essenciais para depuração, identificação de gargalos de desempenho e monitoramento da saúde do sistema.
- **Banco de Dados (Databases):** Cada microsserviço pode ter seu próprio banco de dados otimizado para suas necessidades específicas (e.g., relacional, NoSQL, grafo).

- **Frontend (Single Page Application - SPA):** Uma aplicação cliente (e.g., React, Angular, Vue.js) que se comunica com o Gateway de API para consumir os dados e funcionalidades expostos pelos microsserviços.



10. Visão de Dados (opcional)

N/A - não se aplica.

11. Tamanho e Desempenho

N/A - não se aplica.

12. Qualidade

N/A - não se aplica.

13. Anexos

N/A - não se aplica.

14. Referências

Visão geral do ASP.NET Core MVC - Learn Microsoft:
<https://learn.microsoft.com/pt-br/aspnet/core/mvc/overview?view=aspnetcore-9.0>

Visão de Processos de Negócio - UFMG:
<https://homepages.dcc.ufmg.br/~clarindo/arquivos/disciplinas/uml-mpn/material/transparencias/8-visao-processos-negocio.pdf>

Criar um diagrama de implantação UML - Suporte da Microsoft:
<https://support.microsoft.com/pt-br/topic/criar-um-diagrama-de-implanta%C3%A7%C3%A3o-uml-ef282f3e-49a5-48f5-a6ae-69a6982a4543>

Normas para a segurança da informação: Uma visão geral:
<https://www.dqsglobal.com/pt-br/academy/blog/normas-para-a-seguranca-da-informacao-uma-visao-geral>

Normas técnicas de Usabilidade:
<https://brasil.uxdesign.cc/normas-t%C3%A9cnicas-de-usabilidade-aac4e1576e76>

A maior parte do documento fiz com auxílio da inteligência artificial peguei alguns materiais de referência para fazer o documento

15. Aprovações

Aprovações		
Participante	Assinatura	Data

N/A - não se aplica.